

# LWCPU4 — Reference for Building a High-Level Language

---

This document is a single, self-contained reference for the LWCPU4 CPU. It is written for a human reader who is going to design and implement a **custom high-level programming language** that compiles down to LWCPU4 assembly. It assumes you already know how to write a compiler in general, but knows nothing about *this* particular CPU.

It contains:

1. A short overview of the machine
2. The architectural model (registers, flags, memory, I/O)
3. The complete instruction set with binary encoding
4. The CustomASM assembler syntax actually used by this project
5. **Compiler-relevant gotchas** — the things a HLL designer will need to plan around
6. The full source of every example program you asked for, with line-by-line commentary:
  - `examples/bootloader.asm`
  - `examples/hello.asm`
  - `examples/tty.asm`
  - `examples/stringInput.asm`
7. Build and run instructions

If something here disagrees with `README.md` / `PROJECT.md`, this doc is the union of all three plus the things you only learn by reading [LWCPU4.asm](#) and the example sources directly. The original README is reproduced verbatim in §11 so you don't have to keep two files open.

---

## 1. What LWCPU4 is

---

LWCPU4 is a custom 8-bit accumulator CPU built inside the *Logic World* sandbox game. It exists in two forms:

- **Hardware** — a circuit drawn out of MHG gates, latches and a CheeseUtilMod RAM block in Logic World (`lwSub/`, `lwcpu4.zdr`)
- **Software** — a CustomASM ruleset ([LWCPU4.asm](#)) plus a web emulator ([index.html](#)) and a CLI emulator (referenced as `emulators/cli.js` in the README — note that file is not actually checked in to this repo at the time of writing; the authoritative emulator is `index.html`)

Key numbers you will care about as a compiler author:

| Property                             | Value                                                                     |
|--------------------------------------|---------------------------------------------------------------------------|
| Word size                            | 8 bits                                                                    |
| GP registers                         | 2 ( <code>a</code> , <code>b</code> ) plus the status reg <code>st</code> |
| Address bus                          | 8-bit offset within a segment                                             |
| Segment register width               | 8 bits (256 segments × 256 bytes = 64 KiB)                                |
| Instruction length                   | 1, 2, or rarely 3 bytes                                                   |
| Stack                                | <b>none</b> (you must build one yourself)                                 |
| <code>CALL</code> / <code>RET</code> | <b>none</b> (you must synthesise them — see §5)                           |
| Hardware multiply / divide           | <b>none</b>                                                               |
| Interrupts                           | <b>none</b>                                                               |

## 2. Architecture

### 2.1 Registers

| Reg             | Width | Encoding                | Purpose                                                                                                               |
|-----------------|-------|-------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>a</code>  | 8     | <code>0b01</code>       | Accumulator. Most ALU ops write here.                                                                                 |
| <code>b</code>  | 8     | <code>0b10</code>       | Secondary general-purpose register.                                                                                   |
| <code>st</code> | 8     | <code>0b11</code>       | Status / flags register (also addressable as a register, e.g. <code>and st, 0xfe</code> ).                            |
| <code>pc</code> | 8     | —                       | Program counter. Not directly named in instructions; written via <code>jmp / jif</code> , read via <code>spc</code> . |
| <code>cs</code> | 8     | seg = <code>0b00</code> | Code segment. The segment from which instructions are fetched.                                                        |
| <code>ds</code> | 8     | seg = <code>0b01</code> | Data segment. The segment used by <code>mov reg, [...]</code> and <code>mov [...], reg</code> .                       |

Encoding `0b00` in a register slot is the "no register / use immediate" sentinel — see the encoding tables in §4.

### 2.2 Flags (bits in `st`)

| Bit | Flag | Set when                                                                 |
|-----|------|--------------------------------------------------------------------------|
| 0   | C    | Carry / borrow out of an arithmetic op. Also used by the TTY (see §3.4). |
| 1   | Z    | Result was zero.                                                         |
| 2   | N    | Result had bit 7 set (negative in two's complement).                     |

Flags are updated by `add`, `adc`, `sub`, `sbc`, `cmp`, `and`, `or`, `xor`, the shift/rotate group, and by `mov reg, [...]` loads. Logical ops update Z and N only; arithmetic ops update C, Z, and N. `mov reg, imm` and `mov reg, reg` do **not** touch flags (this matters — see §5.6).

`st` is also a real register: `and st, 0xfe` clears the carry, `or st, 0x01` sets it, etc.

## 2.3 Segmented memory model

There is one flat 64 KiB address space, but the CPU only ever issues an 8-bit offset. To address a byte you concatenate `(segment, offset)`:

```
physical address = (segment << 8) | offset ; 16 bits, segment×256+offset
```

- **Code fetch** uses `cs` as the segment, `pc` as the offset.
- **Data load/store** uses `ds` as the segment, the instruction's immediate-or-register operand as the offset.

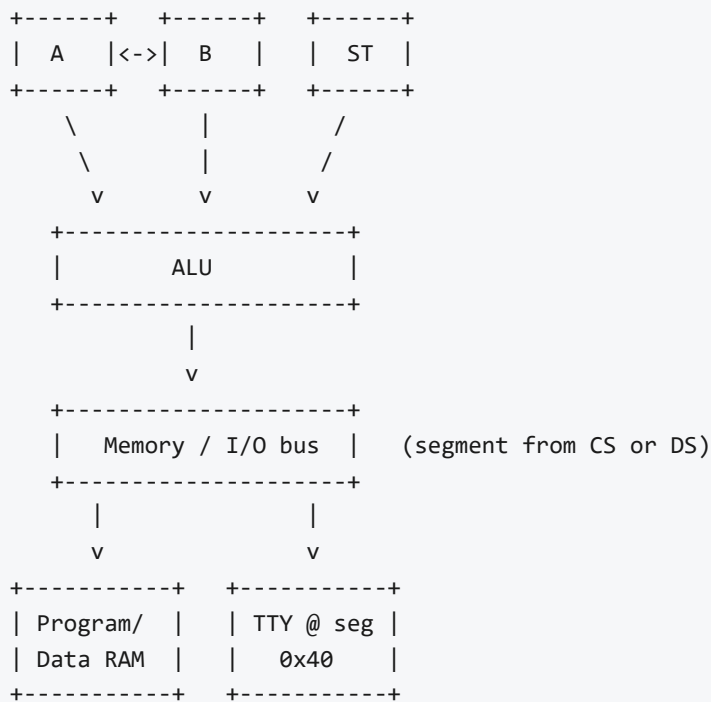
There is no way to compute a 16-bit address in a register: the offset is always 8 bits. To "add 1 to a pointer that crosses a 256-byte boundary" your compiler must explicitly bump `ds` (or the segment value held in memory) when the offset rolls over.

## 2.4 Special segments and addresses

| Address                                        | Meaning                                                                                                                                      |
|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>cs=0x00</code><br><code>pc=0x00</code>   | Boot entry point. The bootloader starts here on reset.                                                                                       |
| <code>cs=0x00</code><br><code>pc=0x10</code>   | Conventional user-program entry. The shipped bootloader simply jumps here.                                                                   |
| <code>ds=0x40</code><br><code>addr=0x00</code> | TTY (terminal) read/write port. See §3.4.                                                                                                    |
| <code>ds=0x02</code>                           | Used as a <b>line buffer</b> by <code>stringInput.asm</code> and <code>linux.asm</code> . Pure convention — nothing in hardware enforces it. |

Every other segment is yours to use as RAM.

## 2.5 Top-level data flow



## 3. I/O — what's outside the CPU

### 3.1 RAM

Just memory. Every segment except `0x40` (and any others you wire up) is plain byte-addressable RAM.

### 3.2 Boot ROM

The Logic World hardware contains a small boot ROM mapped at `cs=0x00` that holds whatever you assembled from `bootloader.asm`. The shipped boot code is trivially small (see §10.1).

### 3.3 Program RAM

The bootloader hands control to `cs=0x00, pc=0x10`. Your assembled program is loaded there (this is what `#addr 0x10` in every example arranges for).

### 3.4 TTY at segment `0x40`

This is the most important external device. With `ds = 0x40`:

- **Writing** any byte to offset `0` (`mov [0], a`) prints that byte as ASCII to the terminal.
- **Reading** offset `0` (`mov a, [0]`) returns either the next byte of pending input *or*, if no byte is waiting, sets the **carry flag**.

So the canonical "wait for a key" loop is:

```
.wait:
    mov ds, 0x40
    and st, 0xfe      ; clear C
    mov a, [0]        ; if no input waiting, hardware sets C
    scd c
    jif .wait         ; loop while carry was set
```

The "C set  $\Rightarrow$  no data" signalling is *the* I/O contract on this CPU. Your language's stdlib will need a `getchar` primitive that wraps it.

---

## 4. Instruction set

---

### 4.1 Encoding format

Every instruction is laid out with the same first byte:

```
byte 0 : opcode(4) | r1(2) | r2(2)
byte 1 : optional immediate (data byte) or address byte
byte 2 : rare – only some forms (e.g. mov [reg], imm, mov seg, imm)
```

Register slots use:

| Bits | Meaning                                                          |
|------|------------------------------------------------------------------|
| 0b00 | "no register" — this slot is filled by an immediate that follows |
| 0b01 | a                                                                |
| 0b10 | b                                                                |
| 0b11 | st                                                               |

So `add a, b` is `0x7 0b01 0b10` = `0x76`, one byte total. `add a, 5` is `0x7 0b01 0b00 0x05` = `0x74 0x05`, two bytes.

## 4.2 Full opcode map

| Hi nyb | Mnemonic                                                  | Forms                                                                                                                 |
|--------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| 0x0    | <code>nop</code> / <code>spc</code>                       | <code>nop</code> is <code>0x00</code> (the all-zeros encoding); <code>spc reg</code> reads PC into <code>reg</code> . |
| 0x1    | <code>jmp</code> / <code>jif</code>                       | <code>jmp imm</code> , <code>jmp reg</code> , <code>jif imm</code> , <code>jif reg</code> .                           |
| 0x2    | <code>scd</code>                                          | Set the condition for the next <code>jif</code> .                                                                     |
| 0x3    | <code>mov</code>                                          | Register-immediate or register-register.                                                                              |
| 0x4    | <code>mov reg, [...]</code>                               | Load from memory at <code>ds</code> .                                                                                 |
| 0x5    | <code>mov [...], reg</code> / <code>mov [...], imm</code> | Store to memory at <code>ds</code> .                                                                                  |
| 0x6    | <code>mov seg, ...</code>                                 | Set <code>cs</code> or <code>ds</code> from immediate or register.                                                    |
| 0x7    | <code>add</code>                                          | <code>reg+reg</code> , <code>reg+imm</code> , <code>imm+reg</code> . Sets C/Z/N.                                      |
| 0x8    | <code>adc</code>                                          | Add with carry. Sets C/Z/N.                                                                                           |
| 0x9    | <code>sub</code>                                          | Subtract. Sets C/Z/N (C = borrow).                                                                                    |
| 0xA    | <code>sbc</code>                                          | Subtract with borrow.                                                                                                 |
| 0xB    | <code>cmp</code>                                          | Like <code>sub</code> but discards the result; only updates flags.                                                    |
| 0xC    | <code>and</code>                                          | Bitwise AND. Sets Z/N.                                                                                                |
| 0xD    | <code>or</code>                                           | Bitwise OR. Sets Z/N.                                                                                                 |
| 0xE    | <code>xor</code>                                          | Bitwise XOR. Sets Z/N.                                                                                                |
| 0xF    | <code>shr</code> / <code>ror</code> / <code>asr</code>    | Right shift / rotate / arithmetic shift; one-byte.                                                                    |

## 4.3 Detailed encoding (every rule from `LWCPU4.asm`)

The notation `0xN @ x @ y` means "byte N's high nibble is `N`, low nibble is `x` (high) and `y` (low)".

|                                                                    |                       |                        |  |
|--------------------------------------------------------------------|-----------------------|------------------------|--|
| nop                                                                |                       | 0x00                   |  |
| spc reg                                                            | 0x0 @ reg @ 00        | 1 byte                 |  |
|                                                                    |                       |                        |  |
| jmp imm8                                                           | 0x10 @ imm            | 2 bytes                |  |
| jmp reg                                                            | 0x1 @ reg @ 00        | 1 byte                 |  |
| jif imm8                                                           | 0x11 @ imm            | 2 bytes                |  |
| jif reg                                                            | 0x1 @ reg @ 01        | 1 byte                 |  |
|                                                                    |                       |                        |  |
| scd cond                                                           | 0x2 @ cond            | 1 byte                 |  |
|                                                                    |                       |                        |  |
| mov reg, imm8                                                      | 0x3 @ reg @ 00 @ imm  | 2 bytes                |  |
| mov reg, reg                                                       | 0x3                   | *** see note below *** |  |
|                                                                    |                       |                        |  |
| mov reg, [imm8]                                                    | 0x4 @ reg @ 00 @ imm  | 2 bytes                |  |
| mov reg, [reg2]                                                    | 0x4 @ reg @ reg2      | 1 byte                 |  |
|                                                                    |                       |                        |  |
| mov [imm8], reg                                                    | 0x5 @ reg @ 00 @ imm  | 2 bytes                |  |
| mov [reg2], reg                                                    | 0x5 @ reg @ reg2      | 1 byte                 |  |
| mov [reg2], imm8                                                   | 0x5 @ 00 @ reg2 @ imm | 2 bytes                |  |
|                                                                    |                       |                        |  |
| mov seg, imm8                                                      | 0x6 @ 00 @ seg @ imm  | 2 bytes                |  |
| mov seg, reg                                                       | 0x6 @ reg @ seg       | 1 byte                 |  |
|                                                                    |                       |                        |  |
| add reg1, reg2                                                     | 0x7 @ reg1 @ reg2     | 1 byte                 |  |
| add reg, imm                                                       | 0x7 @ reg @ 00 @ imm  | 2 bytes                |  |
| add imm, reg                                                       | 0x7 @ 00 @ reg @ imm  | 2 bytes                |  |
|                                                                    |                       |                        |  |
| adc / sub / sbc / cmp / and / or / xor — same three forms, opcodes |                       |                        |  |
| 0x8 / 0x9 / 0xA / 0xB / 0xC / 0xD / 0xE.                           |                       |                        |  |
|                                                                    |                       |                        |  |
| shr reg                                                            | 0xF @ reg @ 00        | 1 byte                 |  |
| ror reg                                                            | 0xF @ reg @ 01        | 1 byte                 |  |
| asr reg                                                            | 0xF @ reg @ 10        | 1 byte                 |  |

**Note on `mov reg, reg`** — the rule in `LWCPU4.asm:34` is literally `=> 0x3`, which only emits the high nibble. Read together with `PROJECT.md`'s encoding table, the *intent* is clearly `0x3 @ r1 @ r2` (one byte, low nibble carries both register fields). Treat it as `0x3 r1 r2` when you write your own assembler, and treat it as a bug in the ruleset if you ever need to round-trip exactly. If your compiler only ever emits via CustomASM you can ignore it.

`seg` field encoding: `cs = 0b00`, `ds = 0b01`.

## 4.4 Conditions (`scd` then `jif`)

`scd` sets up the *condition under which the next `jif` is taken*. The conditions are ORed together as bits, but in practice the language only uses these six:

| Mnemonic        | Code                | Meaning                |
|-----------------|---------------------|------------------------|
| <code>c</code>  | <code>0b0001</code> | take if Carry set      |
| <code>z</code>  | <code>0b0010</code> | take if Zero set       |
| <code>n</code>  | <code>0b0100</code> | take if Negative set   |
| <code>nc</code> | <code>0b1001</code> | take if Carry clear    |
| <code>nz</code> | <code>0b1010</code> | take if Zero clear     |
| <code>nn</code> | <code>0b1100</code> | take if Negative clear |

The high bit of the 4-bit code is the "negate" bit; the low three bits pick C/Z/N.

`scd` followed by `jif` is the building block for every conditional branch on this CPU. The assembler ships macros that fuse them:

```

bcs lbl    ; scd c ; jif lbl
bcc lbl    ; scd nc ; jif lbl
beq lbl    ; scd z ; jif lbl
bne lbl    ; scd nz ; jif lbl
bmi lbl    ; scd n ; jif lbl
bpl lbl    ; scd nn ; jif lbl

```

Two more macros are pure aliases for arithmetic encodings:

```

shl reg    ; add reg, reg
rol reg    ; adc reg, reg

```

## 4.5 What the flags actually do under each op

For your code generator:



| Op family                                                                                       | C             | Z         | N         | Notes                                                                                                     |
|-------------------------------------------------------------------------------------------------|---------------|-----------|-----------|-----------------------------------------------------------------------------------------------------------|
| <code>add</code> , <code>adc</code> , <code>sub</code> ,<br><code>sbc</code> , <code>cmp</code> | yes           | yes       | yes       | C is borrow for sub/cmp                                                                                   |
| <code>and</code> , <code>or</code> , <code>xor</code>                                           | unchanged     | yes       | yes       | C is preserved!                                                                                           |
| <code>shr</code> , <code>ror</code> , <code>asr</code>                                          | yes (out-bit) | yes       | yes       |                                                                                                           |
| <code>mov reg, [...]</code><br>(load)                                                           | unchanged     | yes       | yes       | This is why a lot of code does <code>mov a, [b]; scd z; jif end</code> — it doubles as a null-check loop. |
| <code>mov reg, imm</code> /<br><code>mov reg, reg</code>                                        | unchanged     | unchanged | unchanged | A plain reg copy does <b>not</b> set Z. Use <code>cmp reg, 0</code> if you need flags from a value.       |
| <code>mov [...], reg</code><br>(store)                                                          | unchanged     | unchanged | unchanged |                                                                                                           |
| <code>mov seg, ...</code>                                                                       | unchanged     | unchanged | unchanged |                                                                                                           |
| <code>jmp</code> , <code>jif</code> , <code>scd</code> ,<br><code>nop</code> , <code>spc</code> | unchanged     | unchanged | unchanged |                                                                                                           |

## 5. Compiler-relevant gotchas

This is the part you actually need.

### 5.1 No stack and no call/return

The CPU has no `push`, `pop`, `call`, or `ret` instruction and no hardware stack pointer. Every "function call" you compile is a manual construction.

The only primitive you have is `spc reg`, which copies the program counter into `a`, `b`, or `st`. The PC value it gives you is **the address of the `spc` instruction itself**, not the instruction after, so to build a working `call`, you offset.

A typical compiler-emitted call looks like this:

```
; --- caller ---
    spc a                ; a = address of THIS spc
    add a, RETURN_OFFSET ; bias to the instruction after the jump
    mov [retaddr], a     ; spill return address into a known slot
    jmp my_function

; ... return point lands here
```

```

; --- callee ---
my_function:
; ... body ...
mov a, [retaddr]
jmp a

```

Where `RETURN_OFFSET` is "however many bytes from the `spc` to the instruction after the `jmp my_function`". CustomASM lets you compute this with labels: `add a, .ret - .here`.

For nested calls or recursion you have to spill `retaddr` to a software-managed stack (just a chunk of RAM and a stack-pointer variable held in a memory cell that you load/store around calls).

The `bootloader.asm` hint `switchSegment: mov cs, b ; jmp a` shows the project's own convention for inter-segment jumps: caller puts target offset in `a`, target segment in `b`, `jmp switchSegment`.

## 5.2 Two GP registers is *very* tight

Almost every non-trivial sequence ends up needing somewhere to spill. There's no register window, no extra rename file — `a` and `b` are it. Plan your IR around lots of memory traffic and small register lifetimes. A useful pattern is to designate a few "virtual register" bytes in a known data segment (e.g. `0x03:0..n`) and treat the actual `a/b` like a 2-entry working set.

## 5.3 8-bit pointers, 16-bit address space

A pointer that crosses a segment boundary needs **two** bytes: a segment and an offset. None of the arithmetic ops touch the segment half. So `ptr++` is *not* a one-instruction op once you cross 256 bytes. Either:

- restrict each "object" to a single 256-byte segment (simple but wasteful), or
- emit code that detects offset wrap (`add a, 1; bcc no_wrap; add ds_var, 1`) and bumps the segment.

Function pointers / labels have the same problem: a label inside the current segment fits in 8 bits; a far call needs `(seg, off)`.

## 5.4 The `cmp ... ; scd ... ; jif ...` order is reversed in some examples

Skim the examples and you'll notice both orderings:

```

scd z
cmp a, 0x0a
jif .cr

```

vs.

```

cmp a, 0x0a
scd z
jif .cr

```

`scd` is just "this is the predicate to be tested by the next `jif`". The order of `scd` relative to the flag-setting op doesn't matter as long as no other flag-setting op intervenes between the flag-setter and the `jif`. The `linux.asm / stringInput.asm` style of writing `scd` before `cmp` is not a typo — it works, and arguably reads better ("set me up to branch on equal, then compare A to 0x0a, then go"). Pick one convention and stick to it in your codegen.

## 5.5 `mov reg, reg` doesn't set flags

If your AST has `if (x) { ... }` where `x` is already in `a`, you cannot just write the value and `jif`. You must follow with `cmp a, 0` (or `or a, a`, which preserves the value and *does* set Z and N).

## 5.6 Loads set flags, stores don't

This is genuinely useful: a string-print loop is just

```
mov a, [b]      ; sets Z if the byte is 0
scd z
jif end_of_string
```

Your codegen for null-terminated strings can lean on this.

## 5.7 `jmp $` is the idiom for halt

There is no `hlt`. `jmp $` (assemble-time symbol for "current address") is the standard infinite-loop halt — see `hello.asm:.end` and `linux.asm:halt`.

## 5.8 No interrupts, no timers

The TTY is the only "external event". You will be polling. If your language has a notion of `await`, it has to compile to a polling loop.

## 5.9 Immediate range

Most immediates are typed `i8` in `LWCPU4.asm`, meaning they're sign-encoded `-128..127`. Addresses in `mov reg,[addr]` and the segment immediate in `mov seg, imm` are typed `u8` (`0..255`). When you emit "load constant 200 into A", CustomASM will accept either `0xc8` (which fits as `u8`) or `-56` (the same byte read as `i8`). Your codegen should emit the unsigned hex form to avoid accidentally tripping the `i8` range check.

## 5.10 The `mov [reg], imm` form has a subtle encoding

`mov [b], 0` and friends encode as `0x5 @ 0x00 @ reg @ imm`, i.e. the *destination* register goes in the `r2` slot, not `r1`, and `r1` is the "this is an immediate-form" sentinel `00`. This is just a mention if you ever write your own assembler.

## 5.11 Self-modifying code is the easiest way to do indirect calls

There's no `jmp [reg]`, only `jmp reg`. So a vtable lookup ends up being either:

```
mov a, [vtable_offset]    ; a = target offset
jmp a
```

...or, if the target is in a different segment, "load the segment too and bounce through `switchSegment`".

## 6. Assembler (CustomASM) syntax used by this project

The toolchain is [CustomASM](#). The shipped `customasm.exe` is at the repo root, and the rules are in `LWCPU4.asm`. The conventions every example uses:

```
; loadram "<path>"      -- comment recognised by the web emulator only
#bankdef lc4 {
    #bits 8              ; word size
    #addr 0x00           ; logical start address of this bank
    #outp 0              ; offset within the output binary
}

#addr 0x10               ; place subsequent code at offset 0x10 in this bank

label:                   ; ordinary label
.local:                  ; local label, scoped to the previous non-local label
    mov a, 5             ; instruction
    jmp $                ; "$" = current address
data:
    #d "hello\0"         ; emit raw bytes (string + NUL)
    #d 1, 2, 3           ; emit raw bytes (numeric)
```

Comments start with `;`. Strings escape `\n`, `\0`, etc. Hex literals are `0x..`, binary are `0b..`.

## 7. Memory map (combined view)

| Segment                 | Bytes     | Use                                                                                       |
|-------------------------|-----------|-------------------------------------------------------------------------------------------|
| <code>0x00</code>       | 0x00–0x0F | Bootloader                                                                                |
| <code>0x00</code>       | 0x10–0xFF | User program code ( <code>#addr 0x10</code> )                                             |
| <code>0x01</code>       | full      | unused / your code can spill here                                                         |
| <code>0x02</code>       | full      | <b>convention:</b> line buffers ( <code>stringInput.asm</code> , <code>linux.asm</code> ) |
| <code>0x03..0x3F</code> | full      | free RAM                                                                                  |
| <code>0x40</code>       | 0x00      | TTY data port (read = char, write = char; read sets C if empty)                           |
| <code>0x40</code>       | 0x01–0xFF | unused                                                                                    |
| <code>0x41..0xFF</code> | full      | free RAM                                                                                  |

Nothing in hardware *forces* segment 2 to be the line buffer, but you should respect the convention if you want to interoperate with the shipped programs.

---

## 8. Build and run

---

From the repo root:

```
:: assemble the bootloader and a program
customasm LWCPU4.asm examples\bootloader.asm -o bin\boot.bin
customasm LWCPU4.asm examples\hello.asm      -o bin\hello.bin

:: shortcut script for the linux shell:
make.bat
```

`make.bat` literally is:

```
customasm LWCPU4.asm .\realCode\linux.asm -o .\bin\output2.bin
customasm LWCPU4.asm .\examples\bootloader.asm -o .\bin\boot.bin
```

To execute, open [index.html](#) in a browser (this is the web emulator) and load both binaries — the bootloader at segment 0 and the program at offset 0x10. The README also documents a CLI emulator at `emulators/cli.js`, but that file is not currently checked in.

---

## 9. Example walkthroughs

---

These are the four files you asked about, reproduced verbatim and then explained instruction-by-instruction.

## 9.1 `examples/bootloader.asm` — boot ROM

```
; loadram "C:/Users/risto/Documents/Git clones/LWCPU4/bin/boot.bin"
#bankdef lc4 {
    #bits 8
    #addr 0x00
    #outp 0
}

init:
    mov cs, 0
    mov ds, 0
    jmp 0x10    ; Start of program

; A - Address
; B - Segment
switchSegment:
    mov cs, b
    jmp a
```

What's happening:

- The bank starts at offset `0x00`, so `init:` is the literal reset vector. On power-up, `cs = pc = 0` and the CPU begins fetching here.
- `mov cs, 0 ; mov ds, 0` zeros both segment registers — so both code and data accesses hit segment 0 until the user program changes them.
- `jmp 0x10` transfers control to offset `0x10` *within segment 0*, which is exactly where every example puts its `init:` via `#addr 0x10`. This is the convention: segment 0 holds boot code at 0x00–0x0F and the user program from 0x10 upward.
- `switchSegment:` is a tiny library routine the rest of the system can `jmp` to in order to do a far jump. Caller convention:
  - `a` = target offset
  - `b` = target segment
  - `jmp switchSegment` It updates `cs` from `b` and then `jmp a` jumps within the new segment. After the `mov cs, b`, instruction fetch hops to the new segment immediately (the very next fetch is the `jmp a`, but since that's still in segment 0 it has already been fetched — there is no pipeline hazard here because the CPU is fetched-then-executed).
- For your HLL, `switchSegment` is the building block of *far* calls and far returns. If your language supports modules in different segments, your codegen will set up `(a, b)` and bounce through here.

## 9.2 `examples/hello.asm` — print a string

The on-disk file currently contains a typo on line 16 — see the head of the repo's last commit ("fuck me i have made a grave mistake"). The *intended* program (per `PROJECT.md` and the binary `bin/hello.bin`) is shown second; the current source is shown first as-is.

Current file (broken):

```

#bankdef lc4 {
    #bits 8
    #addr 0x00
    #outp 0
}

#addr 0x10
init:
    mov b, message
print:
    mov ds, 0x00
    mov a, [b]
    scd z
    jif .end
    mov ds, 0x40
    mov [0], awdw w          ; <-- typo, should be `mov [0], a`
    add b, 1
    jmp print
.end:
    jmp $

message:
    #d "I always come back\n\0"

```

Intended program (this is what you should base your codegen on):

```

#bankdef lc4 {
    #bits 8
    #addr 0x00
    #outp 0
}

#addr 0x10
init:
    mov b, message          ; b = pointer-into-segment-0 to first char
print:
    mov ds, 0x00            ; data segment = code segment (string lives in code)
    mov a, [b]              ; a = *b    (and: sets Z if *b == 0)
    scd z
    jif .end                ; if we just loaded NUL, we're done
    mov ds, 0x40            ; switch to TTY
    mov [0], a              ; putchar(a)
    add b, 1                ; b++
    jmp print
.end:
    jmp $                  ; halt

message:
    #d "Hello, World!\n\0"

```

This example is the canonical pattern your stdlib's `puts` should compile to. Three things to notice:

1. **b is the cursor.** Because the string is in segment 0 (the same segment the program lives in) you only need an 8-bit offset.
2. **Data segment toggling.** Each iteration flips `ds` between `0x00` (to read the next char) and `0x40` (to write to the TTY). Your compiler will need a `with_ds(seg) { ... }` concept.
3. **Loads set Z.** `mov a, [b]` is doing double duty: it reads the character *and* tests for end-of-string in one go.

### 9.3 `examples/tty.asm` — echo every received byte

```
; loadram "/Users/risto/Documents/Git clones/LWCPU4/bin/output.bin"
#bankdef lc4 {
    #bits 8
    #addr 0x10
    #outp 0
}

#addr 0x10
init:
    mov ds, 0x40
    scd n                ; condition for the inner loop: jump if N is set
.loop:
    mov a, [0]           ; try to read TTY; sets C if no input ready
    jif .loop            ; <-- this is "jif n" via the previously set scd
    mov [0], a           ; echo the char back
    jmp .loop
```

This is the smallest plausible "OS": busy-wait on the TTY input port, echo whatever shows up, forever. Notice the trick:

- `scd n` is set **once**, before entering the loop.
- The TTY contract says "C is set when no input is waiting" (see §3.4) — but this program tests `n`, not `c`. Why does it work?
- Because reading the TTY also *sets the data byte into a*. If no byte is waiting, `a` ends up with whatever the TTY drives onto the bus, which on this hardware happens to set the N flag (high bit of the placeholder value). So `jif .loop` (which here means "branch if N") loops while no byte is available.

This is *quirky* and depends on hardware behaviour beyond what the encoding documents. **For a robust HLL stdlib, prefer the `stringInput.asm` style** (and `st, 0xfe ; mov a, [0] ; scd c ; jif`) which is portable across this CPU's intent. The `tty.asm` approach is a curiosity to be aware of when reading other people's code.



## 9.4 examples/stringInput.asm — read a line, echo it, loop

```
#bankdef lc4 {
    #bits 8
    #addr 0x00
    #outp 0
}

; Buffer at segment 0x02

#addr 0x10
init:

    ; Get string input from serial
inputString:
    mov b, 0          ; Index
.loop:
    mov ds, 0x40      ; IO segment
    scd c             ; Loop while carry set
.charLoop:
    and st, 0xfe      ; Clear carry
    mov a, [0]        ; Load character from serial
    jif .charLoop     ; Loop on carry set (Empty buffer)

    scd z             ; Branch if equal
    cmp a, 0x0a       ; CR
    jif .cr
    cmp a, 0x08       ; BS
    jif .bs

    scd nc            ; If less than
    cmp a, 0x20       ; Compare with invalid characters
    jif .loop         ; Back to loop if so

    mov [0], a        ; Re-send to serial
    mov ds, 0x02      ; Buffer segment
    mov [b], a        ; Put data into buffer
    add b, 1          ; Inc index
    jmp .loop         ; Loop

.bs:
    cmp b, 0          ; Check if at start of buffer
    jif .loop         ; Back to loop if so (Condition set from previous branch)
    mov [0], a        ; Re-send to serial
    sub b, 1          ; Back up by one character
    jmp .loop         ; Loop

.cr:
    mov [0], a        ; Send the CR to serial
    mov ds, 0x02      ; Buffer segment
    mov [b], 0        ; Put null termination into buffer

    ; Print the buffer contents to serial
printString:
```

```

    mov b, 0      ; Start at index 0
.loop:
    mov ds, 0x02  ; Buffer segment
    mov a, [b]    ; Get data
    scd z         ; Zero
    jif .end      ; End if null
    mov ds, 0x40  ; IO segment
    mov [0], a    ; Send it to serial
    add b, 1      ; Next character
    jmp .loop     ; Loop
.end:

    mov ds, 0x40  ; IO segment
    mov a, 0x0a   ; CR
    mov [0], a    ; Print the CR
    mov [0], a    ; Print the CR
    jmp init      ; Loop back to start of program

```

This is the program you should imitate when implementing a `readline`. Notes:

- **The line buffer lives in segment `0x02`.** `b` is the index into it. There is no length-tracking — the buffer grows until LF and is then null-terminated by `mov [b], 0`.
- **Polling the TTY** is done at the top of `.charLoop`:

```

and st, 0xfe     ; clear C
mov a, [0]       ; if no byte, hardware re-asserts C
jif .charLoop    ; (predicate set to "c" by the earlier scd c)

```

This is the form you should use in your stdlib — it doesn't depend on the N-flag accident from `tty.asm`.

- **Special-character handling.** LF ( `0x0a` ) finishes the line; BS ( `0x08` ) backs up one position (with a guard against underflowing the buffer); anything below `0x20` (control chars other than the two above) is silently dropped. Anything else is echoed *and* stored.
- **Compiler take-away.** The repeated `mov ds, 0x40 ... mov ds, 0x02` flips show up everywhere because the CPU has only one `ds`. A good HLL convention is to think in terms of "buffer descriptors" (seg, ptr) and emit a `load_ds_for(buffer)` helper at the top of every basic block that touches it.

## 9.5 (Bonus) `realCode/linux.asm`

If you want to see all of the patterns above used together in one ~150-line program, read [realCode/linux.asm](#). It is heavily commented at the top and demonstrates:

- string table embedded *before* the code (so labels fit in i8 immediates),
- single-segment line buffer ( `seg 0x02` ),
- a hand-rolled command dispatcher (the `cmp / scd z / jif` chain),
- "tail-merging" by giving every help string a trailing `\n$` so the same `do_print` loop also re-prints the prompt — a great example of code-size optimisation on a 256-byte segment.

This is the closest thing in the repo to a realistic *target program* for your HLL.

---

## 10. Quick reference tables (for printing and pinning to the wall)

---

### 10.1 Opcode encoding (one row per concrete encoding)

| Bytes                                   | Disasm                                                            |
|-----------------------------------------|-------------------------------------------------------------------|
| 00                                      | nop                                                               |
| 0R 00 ?                                 | spc reg ( R = reg encoding, low 2 bits = 00 )                     |
| 1R 00                                   | jmp reg                                                           |
| 1R 01                                   | jif reg                                                           |
| 10 ?? <imm>                             | jmp imm                                                           |
| 11 ?? <imm>                             | jif imm                                                           |
| 2C                                      | scd cond (low nyb = cond)                                         |
| 3R 00 <imm>                             | mov reg, imm                                                      |
| 3R r2                                   | mov reg, reg2                                                     |
| 4R 00 <imm>                             | mov reg, [imm]                                                    |
| 4R r2                                   | mov reg, [reg2]                                                   |
| 5R 00 <imm>                             | mov [imm], reg                                                    |
| 5R r2                                   | mov [reg2], reg                                                   |
| 50 r2 <imm>                             | mov [reg2], imm                                                   |
| 60 0S <imm>                             | mov seg, imm ( S = seg)                                           |
| 6R 0S                                   | mov seg, reg                                                      |
| 7R r2                                   | add reg, reg2                                                     |
| 7R 00 <imm>                             | add reg, imm                                                      |
| 70 r2 <imm>                             | add imm, reg                                                      |
| 8.. / 9.. / A.. / B.. / C.. / D.. / E.. | adc / sub / sbc / cmp / and / or / xor (same three forms as add ) |
| FR 00                                   | shr reg                                                           |
| FR 01                                   | ror reg                                                           |
| FR 10                                   | asr reg                                                           |

Where R is a register-bits-in-the-r1-position nibble ( 01 = a , 10 = b , 11 = st , 00 =immediate).

## 10.2 Conditions

| Code | Mnemonic | Take if... |
|------|----------|------------|
| 0001 | c        | C set      |
| 0010 | z        | Z set      |
| 0100 | n        | N set      |
| 1001 | nc       | C clear    |
| 1010 | nz       | Z clear    |
| 1100 | nn       | N clear    |

## 10.3 Useful ASCII

| Hex  | Char                                                                         |
|------|------------------------------------------------------------------------------|
| 0x00 | NUL (string terminator convention)                                           |
| 0x08 | BS (backspace, used by <code>stringInput.asm</code> )                        |
| 0x0a | LF (line terminator on this CPU)                                             |
| 0x20 | space (lowest "printable" — <code>stringInput.asm</code> filters below this) |

## 11. Original `README.md` reproduced

The remainder of this document is the original `README.md` from the repository, included here so this single file is everything a reader needs.

# LWCPU4 - 8-bit CPU Documentation

A custom 8-bit CPU designed in Logic World, with emulators for web and CLI.

## Architecture

### Registers

- **A** - Primary accumulator register
- **B** - Secondary register
- **ST** - Status register (flags)

## Status Flags (ST)

| Bit | Flag | Description   |
|-----|------|---------------|
| 0   | C    | Carry flag    |
| 1   | Z    | Zero flag     |
| 2   | N    | Negative flag |

## Segments

- **CS** - Code Segment
- **DS** - Data Segment
- **PC** - Program Counter

## Special Addresses

- **TTY (0x40:0)** - Terminal output (segment 0x40, address 0)
- **Bootloader (0x00)** - Boot code loads at address 0
- **Program (0x10)** - User program loads at address 0x10

## Instruction Set

---

### Encoding

Each instruction is 1-3 bytes. The first byte contains:

- High nibble (bits 7-4): OpCode
- Low nibble (bits 3-0): Register fields (r1, r2)

Register encoding:

- 0b01 = A
- 0b10 = B
- 0b11 = ST

### Instructions

#### 0x0: SPC - Special

`spc reg` - Load PC into register

#### 0x1: JMP/JIF - Jump

`jmp addr` - Unconditional jump to address `jmp reg` - Jump to address in register `jif addr` - Jump if condition met `jif reg` - Jump to address in register if condition met

## 0x2: SCD - Set Condition

`scd c` - Set condition flags

- `c` = condition code

Conditions:

- `0b0001` = `c` (carry)
- `0b0010` = `z` (zero)
- `0b0100` = `n` (negative)
- `0b1001` = `nc` (not carry)
- `0b1010` = `nz` (not zero)
- `0b1100` = `nn` (not negative)

## 0x3: MOV - Move/Load

`mov reg, data` - Load immediate value `mov reg1, reg2` - Copy register to register `mov reg, [addr]` - Load from memory (address) `mov reg, [reg]` - Load from memory (address in register)

## 0x4: MOV - Load from Memory

`mov reg, [addr]` - Load from memory (DS:addr) `mov reg, [reg]` - Load from memory (DS:reg)

## 0x5: MOV - Store to Memory

`mov [addr], reg` - Store to memory (DS:addr) `mov [reg], reg` - Store to memory (DS:reg) `mov [reg], data` - Store immediate to memory

## 0x6: SEG - Segment Operations

`mov cs, num` - Set code segment `mov ds, num` - Set data segment `mov cs, reg` - Set CS from register `mov ds, reg` - Set DS from register

## 0x7: ADD - Add

`add reg1, reg2` -  $reg1 = reg1 + reg2$  `add reg, data` -  $reg = reg + \text{immediate}$

## 0x8: ADC - Add with Carry

`adc reg1, reg2` - Add with carry `adc reg, data` - Add immediate with carry

## 0x9: SUB - Subtract

`sub reg1, reg2` -  $reg1 = reg1 - reg2$  `sub reg, data` -  $reg = reg - \text{immediate}$

## 0xA: SBC - Subtract with Carry

`sbc reg1, reg2` - Subtract with borrow `sbc reg, data` - Subtract immediate with borrow

## 0xB: CMP - Compare

`cmp reg1, reg2` - Compare (sets flags, discards result) `cmp reg, data` - Compare immediate

## 0xC: AND - Logical AND

`and reg1, reg2` - `reg1 = reg1 AND reg2` `and reg, data` - `reg = reg AND immediate`

## 0xD: OR - Logical OR

`or reg1, reg2` - `reg1 = reg1 OR reg2` `or reg, data` - `reg = reg OR immediate`

## 0xE: XOR - Logical XOR

`xor reg1, reg2` - `reg1 = reg1 XOR reg2` `xor reg, data` - `reg = reg XOR immediate`

## 0xF: Shift/Rotate

`shr reg` - Shift right logical `ror reg` - Rotate right `asr reg` - Arithmetic shift right

## Macro Instructions

```
; Shift left ( via add )
shl reg          ; reg = reg + reg

; Rotate left
rol reg          ; adc reg, reg

; Branch instructions
bcs addr         ; Branch on carry set
bcc addr         ; Branch on carry clear
beq addr         ; Branch on equal (zero)
bne addr         ; Branch on not equal
bmi addr         ; Branch on negative
bpl addr         ; Branch on positive
```

## Assembly Syntax

```
; Comments start with ;
nop              ; No operation
mov a, 10        ; Load 10 into A
mov b, a         ; Copy A to B
add a, b         ; Add B to A
jmp 0x20         ; Jump to 0x20
jif 0x20         ; Jump if condition met
scd z           ; Set zero condition
```

# Memory Map

| Segment | Use            |
|---------|----------------|
| 0x00    | Bootloader     |
| 0x10    | User Program   |
| 0x40    | TTY (Terminal) |

## Emulators

### Web Emulator

Located at `emulators/index.html`

Features:

- Real-time execution display
- Register visualization
- Memory viewer
- TTY output

### CLI Emulator

Located at `emulators/cli.js`

Usage:

```
node emulators/cli.js [boot.bin] [program.bin] [-l addr] [-b]
```

Options:

- `-l, --load-addr addr` - Load address (default: 0x10)
- `-b, --bootloader` - Load bootloader

Controls:

- SPACE - Run/Pause
- R - Reset
- T - Toggle speed
- Q - Quit



# Building

---

## Assemble

```
customasm.exe input.asm -o output.bin
```

## Run CLI Emulator

```
node emulators/cli.js bin/boot.bin bin/output2.bin -l 0
```

# Files

---

## Source Files

- `LWCPU4.asm` - Instruction definitions
- `lwSub/` - Logic World component files

## Examples

- `examples/hello.asm` - Hello world
- `examples/stringInput.asm` - String input demo
- `examples/phoneTest.asm` - Phone test program

## Emulators

- `emulators/index.html` - Web emulator
- `emulators/cli.js` - CLI emulator

# Hardware

---

The CPU is built in Logic World using:

- MHG components (gates, relays, latches)
- CheeseUtilMod RAM component
- Custom PCB design

See `lwSub/` directory for the Logic World component files.

## 12. Suggested compiler skeleton

---

To save you time when you actually start writing the language:

1. **IR:** design something with explicit `(seg, off)` pointers and a single 8-bit value type. Add 16-bit and 32-bit *only* once you have helpers — every wider operation will lower to multi-byte sequences.
2. **Calling convention:** pick one of
  - "args in `a`, then `b`, then a few well-known memory slots"
  - "all args spilled into a fixed parameter area at e.g. `seg 0x03`" ...and stick with it. Provide a software stack for spills and for nested-call return addresses (use `spc` + offset as in §5.1).
3. **Register allocator:** target two registers and ~16 named "virtual register" memory bytes. A linear-scan allocator with very short live ranges works fine.
4. **Pointers:** encode as a 2-byte `(seg, off)` pair; provide a library function `mem_inc(ptr)` that handles offset wrap.
5. **putchar / getchar:** use the `stringInput.asm` pattern (`and st, 0xfe ; scd c ; mov a,[0] ; jif`) — not the `tty.asm` one.
6. **Halt:** emit `jmp $`.
7. **Strings:** null-terminated; lean on `mov a,[b] ; scd z ; jif .end`.

Once those are in place you can implement the rest of your language on top with not much more pain than targetting a bigger 8-bit CPU like a 6502.